Trong ví dụ sau, để tìm bội số chung nhỏ nhất của hai số nguyên dương a và b chúng ta sẽ sử dụng con trỏ hàm như một biến để truyền cho hàm. Vì bội số chung nhỏ nhất của a và b là thương số của tích a và b cho ước số chung nhỏ nhất của a và b. Do đó, ta hoàn toàn có thể thiết kế được một hàm bao gồm các đối số là a, b và một con trỏ hàm là ước số chung nhỏ nhất của a, b. Chương trình được thể hiện như sau:

Ví dụ 3.21: Tìm bội số chung nhỏ nhất của hai số nguyên dương a, b;

```
#include      <stdio.h>
int     USCLN( int  a, int b) {
        while ( a!=b){
                if(a > b)  a  = a-b;
                else     b = b-a;
        }
        return(a);
}
int     BSCNN( int a, int b, int  (*cmp)() ) {
        int k = cmp(a,b);
        return( (a * b)/k);
}
void main(void) {
        int  a, b, (*p)();
        printf("\n Nhập a ="); scanf("%d", &a);
        printf("\n Nhập b ="); scanf("%d", &b);
        p = USCLN;
        printf("\n BSCNN = %d", BSCNN(a, b, p));
        getch();
}
```

PTIT

6. Dữ liệu kiểu tệp (FILE)

6.1. Thâm nhập vào thư viện chuẩn

Mỗi tệp gốc có tham trỏ tới hàm thư viện chuẩn đều phải chứa dòng khai báo

#include < tên_tệp_thư_viện >

ở đầu tệp văn bản chương trình. Dấu ngoặc nhọn < **tên_tệp_thư_viện** > thay cho dấu nháy thông thường để chỉ thị cho trình biên dịch tìm kiếm tệp trong danh mục chứa thông tin tiêu đề chuẩn được lưu trữ trong thư mục include. Trong trường hợp chúng ta sử dụng kí tự "**tên_tệp_thư_viện**" trình biên dịch sẽ tìm kiếm tệp thư viện tại thư mục hiện tại.

Chỉ thị #include "**tên_tệp_thư_viện**" còn có nhiệm vụ chèn thư viện hàm của người sử dụng vào vị trí của khai báo. Trong ví dụ sau, chúng ta sẽ viết chương trình thành 3 tệp, tệp define.h dùng để khai báo tất cả các hằng sử dụng trong chương trình, tệp songuyen.h dùng khai báo nên nguyên mẫu của hàm và mô tả chi tiết các hàm giống như một thư viện của người sử dụng, tệp mainprg.c là chương trình chính ở đó có những lời gọi hàm từ tệp songuyen.h.

Ví dụ 6.1: Xây dựng một thư viện đơn giản mô tả tập thao tác với số nguyên bao gồm: tính tổng, hiệu, tích, thương, phần dư, ước số chung lớn nhất của hai số nguyên dương a, b.

```
/* Nội dung tệp define.h */
#include    <stdio.h>
#include    <conio.h>
#include    <dos.h>
#define     F1    59
#define     F2    60
#define     F3    61
#define     F4    62
#define     F5    63
#define     F6    64
#define     F1    65
#define     F10   68
/* Nội dung tệp songuyen.h */
void   Init_Int( int *, int *);
int    Tong_Int(int , int );
int    Hieu_Int(int, int );
int    Tich_Int(int, int );
float  Thuong(int , int );
int    Mod_Int(int, int);
int    UCLN(int , int);
void   Init_Int( int *a, int *b ){
       printf("\n Nhập a = "); scanf( "%d", a);
```

PTIT

```
        printf("\n Nhập b = "); scanf( "%d", b);
}
int     Tong_Int( int a, int b ){
        printf("\n Tổng a + b =%d", a + b);
        delay(2000);
        return(a+b);
}
int     Hieu_Int( int a, int b ){
        printf("\n Hiệu a - b =%d", a - b);
        delay(2000);
        return(a - b);
}
int     Tich_Int( int a, int b ){
        printf("\n Tích a * b =%d", a * b);
        delay(2000);
        return(a*b);
}
float   Thuong_Int( int a, int b ){
        printf("\n Thương a / b =%6.2f", (float) a /(float) b);
        delay(2000);
        return((float) a /(float) b);
}
int     Mod_Int( int a, int b ){
        printf("\n Phần dư a % b =%d", a % b);
        delay(2000);
        return(a%b);
}
int     UCLN( int a, int b) {
        while (a != b) {
                if ( a > b) a -=b;
                else    b-=a;
        }
        printf("\n UCLN =%d", a);
```

PTIT

```
        delay(2000); return(a);
}
/* Nội dung tệp mainprg.c */
#include        "define.h"
#include        "songuyen.c"
void thuchien(void){
        int a, b, control = 0; char c; textmode(0);
        do {
                clrscr();
                printf("\n TẬP THAO TÁC VỚI SỐ NGUYÊN");
                printf("\n F1- Nhập hai số nguyên");
                printf("\n F2- Tổng hai số nguyên");
                printf("\n F3- Hiệu hai số nguyên");
                printf("\n F4- Tích hai số nguyên");
                printf("\n F5- Thương hai số nguyên");
                printf("\n F6- Phần dư hai số nguyên");
                printf("\n F7- UCLN hai số nguyên");
                printf("\n F10- Trở về");
                c = getch();
                switch(c) {
                        case F1: Init_Int(&a, &b); control =1; break;
                        case F2:
                                if (control) Tong_Int(a, b);
                                break;
                        case F3:
                                if (control) Hieu_Int(a, b);
                                break;
                        case F4:
                                if (control) Tich_Int(a, b);
                                break;
                        case F5:
                                if (control) Thuong_Int(a, b);
                                break;
```

```
                    case F6:
                        if (control) Mod_Int(a, b);
                        break;
                    case F7:
                        if (control) UCLN_Int(a, b);
                        break;
                }
        } while (c!=F10);
}
void main(void) {
        thuchien();
}
```

6.2. Thâm nhập tệp

Các chương trình được viết ở trên tất cả đều đọc vào từ thiết bị vào chuẩn và đưa ra trên thiết bị ra chuẩn, các thiết bị này được ta giả thiết là đã có sẵn cho chương trình và do Hệ điều hành phân bổ.

Bước tiếp theo trong việc xét vào và ra là viết một chương trình thâm nhập tới tệp vốn chưa được nối với chương trình. Chương trình cat sau đây sẽ minh hoạ cho nhu cầu cần tới các thao tác đó. Cat sẽ ghép một tập có tên và đưa kết quả ra thiết bị chuẩn. Cat thường được dùng để in các tệp trên màn hình và có thể xem nó như một bộ nhập vạn năng dùng cho các chương trình không có khả năng thâm nhập tệp theo tên. Chẳng hạn lệnh

```
cat        x.c      y.c
```

sẽ in nội dung các tệp x.c và y.c lên thiết bị ra chuẩn.

Để đọc được các tệp có tên mà người sử dụng vẫn coi chúng là các câu lệnh thực tế để đọc dữ liệu thì trước khi đọc hay ghi, tệp phải được mở bằng hàm thư viện chuẩn fopen. fopen nhận một tên ngoài giống như x.c hoặc y.c, và thực hiện phân tích cú pháp tên sau đó trao đổi với hệ điều hành rồi cho một tên nội bộ để dùng về sau khi đọc hoặc ghi lên tệp.

Tên nội bộ này, trong thực tế, là con trỏ, được gọi là con trỏ tệp, trỏ tới cấu trúc chứa thông tin về tệp chẳng hạn như vị trí bộ đệm, vị trí kí tự hiện tại trong bộ đệm, tệp đang được đọc hay ghi và các thông tin cơ bản nhất về file. Người sử dụng không cần biết các chi tiết vì trong stdio.h có một phần các định nghĩa vào và ra chuẩn đã xác định các chi tiết trên trong định nghĩa cấu trúc được gọi là FLLE. Khai báo duy nhất cần tới cho con trỏ tệp được minh hoạ bởi khai báo

```
FLLE  *fp;
```

Điều này nói lên rằng fp là con trỏ tới FLLE, và fopen cho con trỏ tới FLLE. Lưu ý rằng FLLE là tên kiểu, như int, chứ không phải là cấu trúc, nó đã được cài đặt bằng chỉ thị typedef. Lời gọi tới fopen trong chương trình là

fp = fopen(name, mode);

Đối thứ nhất của fopen là tên của tệp, có dạng một xâu kí tự. Đối thứ hai là mode, cũng là xâu kí tự chỉ ra cách ta định sử dụng tệp. Các mốt sử dụng trong khi mở tệp là:

Mode xử lý	ý nghĩa
"r"	Mở file đã tồn tại để đọc (read only)
"w"	Mở file mới để ghi. Nếu file đã tồn tại, nội dung của nó sẽ bị loại bỏ để thay vào đó nội dung mới
"a"	
"r+"	
"w+"	Mở file mới để ghi và đọc. Nếu file đang tồn tại nội dung của nó sẽ bị huỷ để thay vào nội dung mới.
"a+"	Mở file đã có hoặc tạo nên file mới để thực hiện đọc hoặc ghi thêm dữ liệu vào cuối file.
"rb"	Mở file nhị phân đã tồn tại để thực hiện đọc
"wb"	Mở file nhị mới để ghi. Nếu file đang tồn tại, nội dung của file bị xoá bỏ để ghi vào thông tin mới
"ab"	Mở file nhị phân đang tồn tại để ghi thêm dữ liệu vào cuối file. Nếu file chưa tồn tại, một file mới sẽ được tạo ra
"r + b"	Mở file nhị phân mới cho cả đọc và ghi. Nếu file đang tồn tại thì nội dung của nó bị xoá bỏ và thay thế vào đó nội dung mới
"a + b"	Mở file nhị phân đang tồn tại và ghi thêm dữ liệu vào cuối file. Nếu file chưa tồn tại thì một file mới sẽ được tạo ra.

Nếu mở một tệp chưa tồn tại để ghi hoặc hiệu đính thì nó sẽ được tạo ra (nếu có thể). Mở một tệp đã có để ghi làm cho nội dung cũ sẽ bị mất. Cố đọc một tệp chưa có là một lỗi, và có thể có các nguyên nhân khác cũng tạo ra lỗi (giống như việc cố đọc một tệp không được phép đọc). Nếu có lỗi nào đó thì fopen sẽ cho con trỏ NULL.

Có nhiều phương pháp để đọc hoặc ghi, trong đó dùng fgetc để đọc một kí tự và fputc để đưa ra một kí tự là đơn giản nhất, fgetc sẽ cho kí tự tiếp theo từ tệp; nó cần tới con trỏ tệp để xác định tệp. Vậy

c = fgetc(fp)

sẽ đặt vào c kí tự tiếp từ tệp, được trỏ bởi fp và sẽ đặt EOF khi đạt tới cuối tệp. fputc là ngược lại của fgetc

fputc(c, fp)

ghi kí tự c lên tệp fp và cho kết quả c. Giống getchar và putchar, getc và putc có thể là các macro chứ không phải là hàm.

Khi chương trình chạy, ba tệp sẽ được tự động mở ra và mỗi tệp được gán một con trỏ tệp. Ba tệp này là tệp vào chuẩn và tệp ra chuẩn và tệp báo lỗi chuẩn; các con trỏ tệp tương ứng được gọi là stdin, stdout và stderr.

getchar và putchar có thể được xác định dưới dạng getc, putc, stdin và stdout như sau:

```
#define getchar() getc(stdin)
#define putchar(c) putc(c, stdout)
```

Đối với các tệp vào hoặc ra có khuôn dạng, có thể sử dụng các hàm fcanf và fprintf. Các hàm này đồng nhất với scanf và printf trừ ra đối thứ nhất là con trỏ tệp xác định ra tệp phải đọc hoặc ghi; xâu điều khiển là đối thứ hai.

Ví dụ 6.2: Đưa nội dung của các tệp ra thiết bị ra chuẩn.

```
#include <stdio.h>
void filecopy (FILE *fp);
void main (agrc, agrv) /*cat: ghép nối tệp*/
int agrc;
char *agrv[ ];
{
        FLLE *fp, *fopen();
        if (agrc = = 1) /*không đối: sao cái vào chuẩn*/
            filecopy(stdin);
        else
            while (--argc > 0)
                if((fp = fopen(*+ +argv, "r")) = = NULL) {
                    printf("cat: không mở được tệp%s\ n", *argv);
                    break;
                } else {
                    filecopy(fp);
                    fclose(fp);
                }
}
void filecopy(FILE *fp) /*sao tệp fp lên thiết bị ra*/
{
        int c;
```

PTIT

```
        while ((c = fgetc(fp)) ! = EOF)
                putc(c, stdout);
    }
```

Các con trỏ tệp stdin stdout đã được xác định trước trong thư viện vào / ra chuẩn; chúng có thể được dùng ở bất kì ở đâu có đối tượng kiểu FILE xuất hiện. Tuy nhiên, chúng là các hằng chứ không phải là biến.

Hàm fclose là hàm ngược lại của fopen; nó phá vỡ mối liên hệ giữa con trỏ tệp và tên ngoài đã được thiết lập bởi fopen, giải phóng con trỏ tệp để dùng cho tệp khác. Vì hầu hết các Hệ điều hành đều giới hạn số các tệp mở ra đồng thời nên chúng ta chú ý giải phóng các tệp khi không cần sử dụng.

6.3. Xử lý lỗi - Stderr và Exit

Việc xử lý lỗi trong ví dụ cat ở trên không phải là tốt vì nếu một trong các tệp không thể thâm nhập được bởi một lý do nào đó thì thông báo chỉ được in ra ở cuối của kết quả ghép nối. Điều này là chấp nhận được nếu thiết bị vào ra là màn hình, nhưng vấn đề sẽ xấu đi nếu dự thiết bị vào ra là tệp hoặc một chương trình khác thông qua đường ống.

Để xử lý tốt hơn tình huống này, chúng ta gán thêm tệp ra thứ hai, được gọi là stderr, cho chương trình theo cùng cách như đã làm với stdin và stdout. Nếu có thể được thì output được ghi lên stderr cũng sẽ xuất hiện trên màn hình của người sử dụng mặc dù thiết bị ra chuẩn đã được hướng theo lối khác không là màn hình.

Ví dụ 5.3: Cải biên cat.c để viết thông báo lỗi cho nó trên tệp lỗi chuẩn

```
#include <stdio.h>
void filecopy(FILE *fp) /*sao tệp fp lên thiết bị ra*/
{
        int c;
        while ((c = fgetc(fp)) ! = EOF)
                putc(c, stdout);
}
void main (argc, argv) /*cat: ghép nối tệp*/
int argc;
char *argx[ ];
{
        FILE *fp, fopen();
        if (argc = = 1) /*không có đối, sao ra thiết bị chuẩn*/
           filecopy(stdin);
        else
```

```
        while(--argc > 0)
            if((fp = fopen(*+ + argv, "r")) = = NULL) {
                fprintf(stderr,"cat: không mở được tệp
                %s \ n"), *argv);
                exit(1);
            } else {
                filecopy(fp);
                fclose(fp);
            }
    exit(0);
}
```

Chương trình báo lỗi theo hai cách. Chẩn đoán do fprintf tạo ra được ghi lên stderr cho nên có cách để đưa ra màn hình của người sử dụng thay vì bị mất hút trong đường ống hoặc ghi lên tệp ra.

Chương trình cũng có thể sử dụng hàm thư viện chuẩn exit để kết thúc việc thực hiện. Đối của exit thích hợp cho mọi quá trình gọi tới nó cho nên ta có thể kiểm tra được sự thực hiện chương trình có thuận buồm xuôi gió hay không bằng một chương trình khác sử dụng nó như một tiến trình con. Theo quy ước giá trị 0 do exit cho lại báo hiệu mọi sự diễn ra trôi chảy, còn các giá trị khác 0 báo hiệu những tình huống bất thường.

6.4. Đưa vào và đưa ra cả dòng

Thư viện chuẩn cung cấp trình fgets để đọc một dòng từ tệp. Lời gọi tới fgets được thực hiện thông qua:

fgets(line, MAXLINE, fp)

đọc vào bảng kí tự line (kể cả dấu xuống dòng) từ tệp fp nhiều nhất là MAXLINE - 1 kí tự. Dòng kết quả line được kết thúc bởi '\ 0' còn khi gặp cuối tệp nó sẽ cho NULL.

Để đưa ra cả dòng, hàm fputs sẽ ghi xâu (không nhất thiết chứa dấu xuống dòng) lên tệp;

fputs(line, fp)

trong đó line là bảng các kí tự, fp là một con trỏ file.

Ví dụ 6.4: Chuyển nội dung của tệp văn bản này thành nội dung của tệp văn bản khác.

```
#include <stdio.h>
#include <dos.h>
void filecopy(char *name1, char *name2){
```

```
        FILE *fp1, *fp2; char str[256];
        int n,i;
        fp1 = fopen(name1,"r");fp2 = fopen(name2,"w");
        if (fp1==NULL){
                printf("\n File khong ton tai");
                delay(2000); return;
        }
        while (fgets(str, 255, fp1)!=NULL){
                fputs(str, fp2);
        }
        fclose(fp1); fclose(fp2);
}
void main(void){
        char name1[12], name2[12];
        printf("\n Nhap ten file1:"); scanf("%s",name1);
        printf("\n Nhap ten file2:"); scanf("%s",name2);
        filecopy(name1, name2);
}
```

6.5. Đọc và ghi file bằng fread, fwrite

Đối với những file không được tổ chức thành từng dòng như file của các số nguyên, file của các số thực hoặc file của các cấu trúc, việc đọc và ghi file sẽ không thể thực hiện được bằng fgets, fputs. Chúng ta có thể dùng hàm fread(), frwite() trong thư viện chuẩn của C để đọc hoặc ghi từng block thông tin trong file.

size_t fread (void *ptr, size_t size, size_t n, FILE *fp);

fread() đọc vào con trỏ ptr n phần tử mỗi phần tử có kích cỡ n byte từ tệp được trỏ bởi con trỏ file fp, hàm trả lại số phần tử size_t thực sự được đọc.

size_t fwrite (void *ptr, size_t size, size_t n, FILE *fp);

fwrite() ghi n phần tử trong đó kích cỡ của mỗi phần tử là size byte từ con trỏ ptr vào tệp được trỏ bởi con trỏ file fp.

Trong ví dụ sau, chúng ta sẽ thực hiện hai công việc ghi n cấu trúc sinhvien vào tệp data.dat sau đó sử dụng fread đọc nội dung của data.dat.

Ví dụ 6.5: Ghi n cấu trúc sinhvien vào tệp và đọc cấu trúc sinhvien từ tệp.

```
#include    <stdio.h>
#include    <string.h>
#include    <io.h>
```

```
typedef struct {
        int ngay;
        int thang;
        int nam;
} date;
typedef struct {
        int masv;
        char hoten[30];
        date ngaysinh;
        float dki;
        float dkii;
        float dtk;
        char xeploai[12];
} sinhvien;
void Fwrite_Struct(char *);
void Fread_Struct(char *);
void Fwrite_Struct(char *name){
        sinhvien sv; FILE *fp;char c='y';
        float t;clrscr();
        fp=fopen(name,"w");
        if(fp==NULL){
                printf("\n Khong tao duoc tep");
                delay(2000); return;
        }
        while (c) {
                printf("\n Ma sinh vien:"); scanf("%d",&sv.masv);
                printf("\n Ho va ten:");fflush(stdin);gets(sv.hoten);
                printf("\n Ngay sinh:");scanf("%d", &(sv.ngaysinh.ngay));
                printf("\n Thang sinh:");scanf("%d", &(sv.ngaysinh.thang));
                printf("\n Nam sinh:");scanf("%d", &(sv.ngaysinh.nam));
                printf("\n Diem ky i:");scanf("%f", &t);sv.dki = t;
                printf("\n Diem ky ii:");scanf("%f", &t);sv.dkii = t;
                (sv.dtk) = ((sv.dki) + (sv.dkii))/2;
```

PTIT

```
                printf("\n Diem tong ket:%6.2f",sv.dtk);
                if ( (sv.dtk)>=8.5)    strcpy(sv.xeploai, "Gioi");
                else if((sv.dtk)>=7.0 && (sv.dtk)< 8.5 )
                        strcpy(sv.xeploai, "Kha");
                else if((sv.dtk)>=5.0 && (sv.dtk)< 7.0 )
                        strcpy(sv.xeploai, "Trung binh");
                else
                        strcpy(sv.xeploai, "Yeu");
                printf("\n Xep loai sinh vien:%s", sv.xeploai);
                fwrite(&sv,sizeof(sinhvien), 1, fp);
                printf("\n\n Tiep tuc (Y/N):"); c=getch();
                if( tolower(c)=='n') break;
                else clrscr();
        }
        fclose(fp);
}
void Fread_Struct(char *name){
        sinhvien sv; FILE *fp;long int n, i=0;
        fp=fopen(name,"r");
        if(fp==NULL){
                printf("\n Khong tao duoc tep");
                delay(2000); return;
        }
        clrscr(); /* Tính độ dài file */
        n=filelength(fileno(fp))/sizeof(sinhvien);
        while (i<n) {
                fread(&sv, sizeof(sinhvien), 1, fp);
                printf("\n");
                printf("%-5d", sv.masv);
                printf("%-20s", sv.hoten);
                printf("%2d/", sv.ngaysinh.ngay);
                printf("%2d/",sv.ngaysinh.thang);
                printf("%2d",sv.ngaysinh.nam);
```

PTIT

```
            printf("%6.2f",sv.dki);
            printf("%6.2f",sv.dkii);
            printf("%6.2f",sv.dtk);
            printf("%10s", sv.xeploai);
            i++;
        }
        getch();fclose(fp);
}
void main(void) {
        char name[]="ketqua.dbf";
        Fwrite_Struct(name);
        Fread_Struct(name);
}
```

6.6. Đọc và ghi file bằng fscanf, fprintf

Khác với fread và fwrite, fscanf đọc dữ liệu theo định dạng từ tệp và fprintf ghi thông tin theo định dạng vào tệp

```
fprintf(fp, formats, available_list);
fscanf(fp, formats, available_list);
```

trong đó fp là một con trỏ file, formats là danh sách các xâu định dạng như %d, %s, %f . . ., tương ứng với các xâu định dạng là danh sách các biến được mô tả trong available_list. Ví dụ về đọc và ghi ma trận vào file sẽ minh hoạ về cách sử dụng fprintf và fscanf.

Ví dụ 6.6: Cho file dữ liệu INPUT.TXT được tổ chức như sau:

Dòng đầu tiên ghi lại một số nguyên dương n là cấp của ma trận vuông. N dòng tiếp theo mỗi dòng ghi n số thực, mỗi số thực được phân cách với nhau bởi một vài dấu trống. Hãy tìm tính định thức của ma trận vuông trong file dữ liệu INPUT.TXT và ghi lại kết quả vào file OUTPUT.TXT, nếu định thức bằng 0 hãy ghi lại thông báo "Định thức D=0".

Ví dụ về file INPUT.TXT

```
3
1    2    4
4    8    12
3    -3   0
```

Khi đó định thức được tính là -36 được ghi lại trong file OUTPUT.TXT

Định thức D:= -36

```
#include      <stdio.h>
#include      <conio.h>
#include      <dos.h>
#include      <alloc.h>
/* Khai báo nguyên mẫu cho hàm */
void    Init_Matrix(float **A, int *n);
void   In_Matrix(float **A, int n);
int     Chuyen(float **A, int n, int i);
float   Det_Matrix(float **A, int n);
void    Init_Matrix( float **A, int *n) {
        int i, j; float temp;
        FILE *fp;
        fp= fopen("INPUT.TXT", "r");
        fscanf(fp,"%d",n);
        if (fp==NULL) {
                printf("\n Chua co file du lieu");
                delay(2000);return;
        }
        for (i=0; i<*n;i++)
                A[i]= malloc(*n * sizeof(float));
        for(i=0;i<*n;i++){
                for(j=0;j<*n; j++){
                        fscanf(fp,"%f",&temp);
                        A[i][j]=temp;
                }
        }
        fclose(fp);
        In_Matrix(A,*n);
}
void    In_Matrix( float **A, int n){
        int i, j;
        for(i=0;i<n;i++){
```

PTIT

```
                printf("\n%-3c", 179);
                for(j=0;j<n; j++){
                        printf("%8.2f", A[i][j]);
                }
                printf("%3c", 179);
        }
        delay(2000);
}
int chuyen(float **A, int n, int i){
        float j, k, temp;
        for(j=i+1; j<n;j++){
                if(A[j][i]){
                        for(k=0;k<n;k++){
                                temp=A[i][k];A[i][k]=A[j][k];
                                A[j][k]=temp;
                        }
                        return(1);
                }
        }
        return(0);
}
float Det_Matrix(float **A, int n){
        float D=1, p, x; int i, j, k;
        FILE *fp;
        fp = fopen("OUTPUT.TXT","w");
        if(fp==NULL) {
                printf("\n Khong tao duoc file");
                delay(2000);
                return(0);
        }
        for(i=0; i<n;i++){
                if(A[i][i]==0){
                        if(chuyen(A,n,i)==0){
```

PTIT

```
                        printf("\n Dinh thuc bang 0");
                        fprintf(fp,"%s","Dinh thuc D:=0");
                        fclose(fp);
                        delay(2000);
                        return(0);
                }
                else
                        D =-D;
        }
        for(j=i+1;j<n;j++){
                x = A[j][i]/A[i][i];
                for(k=0; k<n;k++){
                        p = A[i][k] * x;
                        A[j][k]=A[j][k]-p;
                }
                In_Matrix(A,n); clrscr();
        }
    }
    for(i=0;i<n; i++)
        D *=A[i][i];
    printf("\n D=%6.2f", D);
    fprintf(fp,"Dinh thuc D:=%6.2f",D);
    fclose(fp);
    return(D);
}
float **A;
void main(void){
    int i,n;clrscr();
    Init_Matrix(A,&n);
    Det_Matrix(A,n);
    free(A);
}
```

PTIT

6.7. Một số hàm thông dụng khác

Thư viện chuẩn còn đưa ra nhiều hàm nữa, một số trong chúng tỏ ra đặc biệt hữu ích. Ta đã nói tới các hàm xử lý xâu strlen, strcpy, strcat và strcmp. Sau đây là một số hàm khác.

Kiểm tra lớp kí tự và chuyển dạng

Có nhiều macro thực hiện các phép kiểm tra và chuyển dạng kí tự:

Isalpha(c)	Hàm trả lại giá trị khác 0 nếu c là kí tự chữ, bằng 0 nếu ngược lại.
Isupper(c)	Hàm trả lại giá trị khác 0 nếu c là chữ hoa, bằng 0 nếu ngược lại.
Islower(c)	Hàm trả lại giá trị khác 0 nếu c là chữ thường, bằng 0 nếu ngược lại.
isdigit(c)	Hàm trả lại giá trị khác 0 nếu c là chữ số, bằng 0 nếu ngược lại.
isspace(c)	Hàm trả lại giá trị khác 0 nếu c là dấu cách, dấu tab hoặc dấu xuống dòng, bằng 0 nếu ngược lại.
toupper(c)	Chuyển c thành chữ hoa.
telower(c)	Chuyển c thành chữ thường
ungetc(c, fp)	Đẩy kí tự c trở lại tệp fp. Mỗi thời điểm chỉ được phép đẩy lại một kí tự. ungetc có thể dùng được với bất kì hàm vào và macro nào kiểu như scanf, getc, hoặc getchar.

Gọi hệ thống

Hàm system(s) thực hiện chỉ thị lệnh chứa trong xâu kí tự s rồi quay lại thực hiện chương trình hiện tại. Nội dung của s phụ thuộc chặt chẽ vào hệ điều hành cục bộ. Ví dụ

system(“date”)

sẽ cho chương trình date chạy; nó in ra ngày tháng và đợi nhận ngày tháng mới.

Quản lý bộ nhớ

Hàm calloc khá giống với hàm malloc mà ta đã dùng ở các chương trước.

calloc(n, sizeof(*đối tượng*))

cho con trỏ tới vùng không gian đủ lớn để lưu trữ được n đối tượng có kích thước xác định, hoặc cho NULL nếu yêu cầu không được thoả mãn. Phần bộ nhớ sẽ được khởi đầu bằng giá trị 0.

Con trỏ có sự xếp thẳng bộ nhớ đúng cho đối tượng đang xét nhưng phải tạo sắc thái cho nó theo kiểu thích hợp, như trong

```
char *calloc();
int *ip;
ip = (int*) calloc(n, sizeof(int));
```

PTIT

free(p) giải phóng không gian được trỏ bởi p, với ban đầu có được nhờ lời gọi tới calloc. Không có hạn chế gì về trật tự thực hiện việc giải phóng không gian nhưng sẽ bị lỗi nặng nếu giải phóng không gian nào đó không được tạo ra bởi calloc.

Hàm findfirst, findnext tìm file đầu tiên và file kế tiếp trong thư mục được chỉ qua đường dẫn. Hàm trả lại giá trị 0 nếu việc tìm kiếm thành công trả lại giá trị -1 nếu gặp lỗi.

int findfirst(const char *pathname, struct ffblk *ffblk, int attrib);

int findnext(struct ffblk *ffblk);

trong đó char * pathname : là đường dẫn tới thư mục hiện tại, pathname chấp nhận kí tự *, ? thay thế cho xâu kí tự bất kỳ và kí tự bất kỳ. Cấu trúc ffblk gọi là cấu trúc điều khiển thông tin về file của DOS được định nghĩa trong DIR.H

```
struct ffblk {
  char      ff_reserved[21];
  char      ff_attrib;
  unsigned  ff_ftime;
  unsigned  ff_fdate;
  long      ff_fsize;
  char      ff_name[13];
};
```

int attrib; là thuộc tính của file bao gồm:

FA_RDONLY	:	Read-only attribute
FA_HIDDEN	:	Hidden file
FA_SYSTEM	:	System file
FA_LABEL	:	Volume label
FA_DIREC	:	Directory
FA_ARCH	:	Archive

Ví dụ 6.7: Hiển thị tên tất cả các file trên đĩa tại thư mục hiện tại.

```
#include      <stdio.h>
#include      <dir.h>
int main(void) {
  struct ffblk ffblk1;
  int done;
  printf("Directory listing of *.*\n");
  done = findfirst("*.c",&ffblk1,0);
```

PTIT

```
  while (!done){
        printf("  %s     %u\n", ffblk1.ff_name,ffblk1.ff_fdate);
        done = findnext(&ffblk1);
  }
  return 0;
}
```

Hàm fseek: Truy nhập trực tiếp trên file dữ liệu nhị phân

fseek (FILE *fp, long offset, int whence);

FILE *fp : là một con trỏ file

long offset: kích cỡ tính theo byte cần chuyển tới

int whence là một số nguyên được định nghĩa như sau:

SEEK_SET = 0 : di chuyển từ vị trí bắt đầu là đầu tệp

SEEK_CUR = 1 : di chuyển từ vị trí bắt đầu là vị trí hiện tại

SEEK_END = 2 : di chuyển từ vị trí bắt đầu là cuối tệp

Hàm frewin(FILE *fp) : Đặt con trỏ về vị trí đầu tệp

Hàm ferror(FILE *fp): Xác định xem các thao tác trên file có gặp lỗi hay không.

Hàm remove (char *filename): Xoá file được chỉ qua đường dẫn.

Hàm rename(char *oldname, char *newname): Đổi tên file cũ thành tên file mới.

Hàm mkdir(char *path): Tạo lập một thư mục được chỉ qua đường dẫn

Hàm rmdir(char *path): Loại bỏ thư mục được chỉ qua đường dẫn.

Hàm chdir(char *path): Thay đổi thư mục hiện tại

Hàm getcurdir(int driver, char *dir): Nhận thư mục tại ổ đĩa hiện tại

PTIT

TÀI LIỆU THAM KHẢO

[1] Phạm Văn Ất, Kỹ thuật lập trình C, Nhà xuất bản KHKT, 1995.

[2] Quách Tuấn Ngọc, Ngôn ngữ lập tình C, NXB Thống kê, 2003.

[3] Đỗ Xuân Lôi, Cấu trúc dữ liệu và giải thuật, NXB KHKT, 1994.

[4] Nguyễn Duy Phương, Kỹ thuật lập trình, Giáo trình giảng dạy tại Học viện CN-BCVT

[5] Brian Kerninghan, Denis Ritche, C Language. Norm ANSI. Prentice Hall, 1988.

[6] Bryon Gottfried, Programming With C. McGraw Hill, 1996.

[7] Carl Townsend, Understanding C. SAMS, 1989.

[8] Paul Davies, The Inspensable Guide to C. Addision Wisley, 1996.

[9] Nikolus L.R. Wirth, Program = Data Structure + Algorithms. Prentice Hall, 1992.

PTIT